

Dominating Set to Min-Sum Multicenter

Sample

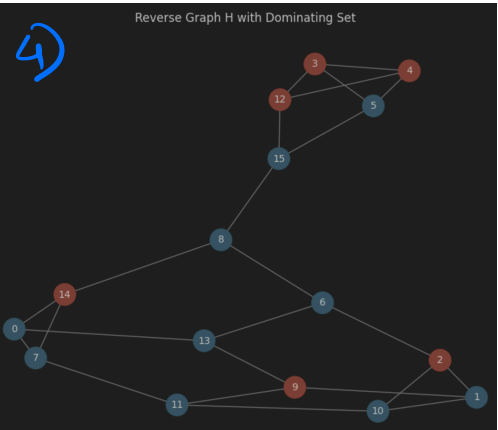
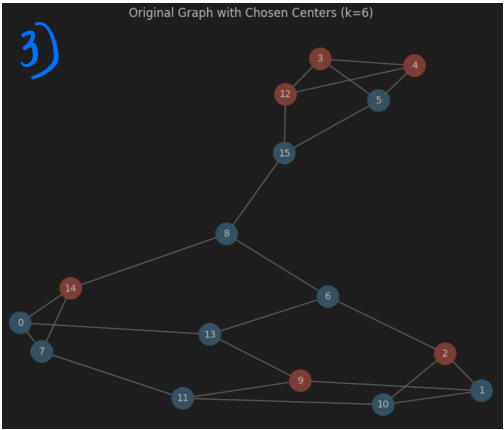
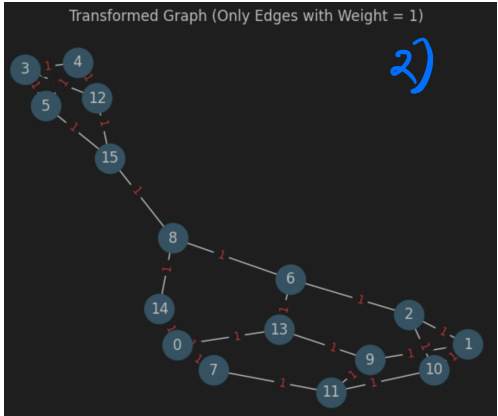
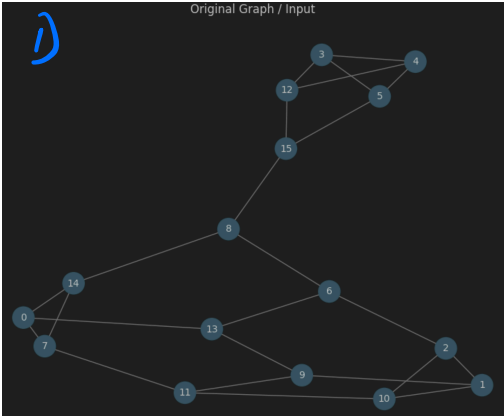
Input:

$k=6$

3	4
3	12
3	5
4	12
4	5
12	15
9	11
9	1
9	13
11	10
11	7
8	15
8	6
8	14
15	5
0	14
0	7
0	13
14	7
1	2
1	10
6	13
6	2
2	10

Transformed Input:

Adjacency List of Transformed Graph G'	
3:	[4, 12, 5]
4:	[3, 12, 5]
12:	[3, 4, 15]
5:	[3, 4, 15]
15:	[12, 5, 8]
9:	[11, 1, 13]
11:	[9, 10, 7]
1:	[9, 10, 2]
13:	[9, 6, 0]
10:	[11, 1, 2]
7:	[11, 14, 0]
8:	[15, 6, 14]
6:	[13, 8, 2]
14:	[7, 8, 0]
0:	[13, 7, 14]
2:	[1, 10, 6]



Step	Time Complexity	Explanation
1. Load Graph from CSV	$O(E)$	Reads each edge or node from the file and adds it to the graph. Time grows linearly with the number of edges.
2. Transform to Min-Sum Multicenter	$O(n^2)$	Builds a complete graph by checking every node pair. Assigns weight 1 if an edge exists, else 999.
3. Compute Min-Sum Cost	$O(k \cdot n)$ per call	For each node, it finds its shortest path to one of the k centers. Repeat this for all non-center nodes.
4. Brute-Force Center Selection	$O(C(n, k) \cdot k \cdot n)$	Tries every combination of k centers and computes the total cost for each. Becomes expensive for large n and k .
5. Verify Dominating Set	$O(n + E)$	Checks if each node is either a center or connected to one. Scans neighbors of all nodes.
6. Visualization	$O(n + E)$ (empirical)	Plots nodes and edges using a layout algorithm. Scales with graph size but is mostly for display.